NeuralSFC 实现与原理总结（2D 落地 + 3D 推广现象）

本文档总结本仓库 NeuralSFC 的端到端实现链路、核心原理、训练时 Generator/Evaluator 的协同机制，以及从 2D 可复现落地到 3D 推广时出现的 matched-portal 卡壳现象与可视化证据。

---

1. 一句话总览（你可以用它复述论文/项目核心）

NeuralSFC 将“离散的像素访问顺序优化”重写为“连续的 dual-graph 边权重学习”：网络输出连续边权重，固定的 MST+splice 解码器将权重确定性地解码为一条合法 SFC，再用真实指标（AC/TV/LZW 等）驱动训练（通过可导代理 Evaluator 解决不可导的离散解码）。

---

2. 端到端数据流：`img → weights → sfc → metric`

2.1 3 个对象

- **`img`**：输入图像（常见为 32×32 灰度，或 3 通道）。
- **`weights`**：dual graph 的边权重向量（不是像素权重）。每条权重对应两个相邻 2×2 block 的“合并选择”。
- **`sfc`**：最终曲线（长度为 $H \cdot W$ 的像素坐标序列），并满足相邻步 $\ell 1=1$（AC）。

2.2 2 个固定函数

- **解码器（固定，不学习）**：`Decode(img, weights) → sfc`
  - 本仓库 2D 落地：
    - `basic/context.py::build_dual_graph`
    - `networkx.minimum_spanning_tree`
    - `basic/context.py::apply_mst`
- **指标函数（固定，不学习）**：`Metric(img, sfc) → y_real`
  - 典型指标实现：
    - `basic/evaluation.py`（AC/TV）
    - `utils/lzw.py`（LZW length）

因此核心链路是：

`img → weights → (MST 解码) → sfc → metric`

---

3. 2D 解码器为什么能得到一条合法 SFC？（工程闭环）

2D 落地版本的关键结构是“2×2 cover + MST merge + cut”。

- **Cover**：把每个 2×2 block 初始化为一个 4-cycle
  - `basic/context.py::build_circuits_graph`
- **Tree**：在 block-level 的对偶图上取 MST
  - `basic/context.py::build_dual_graph` + `networkx.minimum_spanning_tree`
- **Merge（2-edge splice）**：沿 MST 的每条树边，用“删两条 + 加两条”的 splice 合并 cycles
  - `basic/context.py::apply_mst`
- **Cut/Extract**：从最终全局 cycle 剪开一条边抽出 Hamiltonian path

3.1 splice 为什么必须是“删两条 + 加两条”？

因为每个 component 都保持为 2-regular（每个点度为 2）的 cycle。合并两个 cycle 且保持度=2 的最小修改就是：

- 删除每个 cycle 上各一条边（共 2 条）
- 添加两条跨-component 边（共 2 条）

这样 component 数减少 1，且全程不破坏“每点度=2”。

3.2 为什么跨边必须发生在共享 face?

为了满足 AC（相邻点 $\ell 1=1$），跨边 (u,u') 必须是网格的相邻点，因此它只能发生在两个 block 的共享边界（2D 是共享边，3D 是共享面）。

---

4. dafner/context 权重：u+w-e-f(+others) 到底是什么?

4.1 权重的对象是什么?

weights 是 dual graph 上每条 block 邻接边的权重，用于决定 MST 更倾向选哪些 block 邻接关系。

4.2 基础公式（默认/基础版）

在 basic/context.py::calc_weight(..., advanced=False) 和 utils/misc.py::WeightsAssigner.calc_weight 中，核心都是：

- **weight = u + w - e - f**

以“右邻拼接”为例，它构造若干像素差分：

- **u,w**：合并后希望“新增/保留”的跨-block 连接边的像素差（希望小）
- **e,f**：合并时要“断开/删除”的 block 内部边的像素差（希望大）

因此该权重表达的偏好是：

- 在平滑区域更愿意跨块连接
- 在强边界处不愿跨块连接

注意：我们已修复 uint8 溢出问题，使差分在 int 上计算，避免 overflow 警告并保证权重语义正确。

4.3 advanced_weights=True 的差别

advanced=True 会增加额外邻域项 others，将更外围上下文纳入权重；训练管线里的 WeightsAssigner.calc_weight 通常只实现基础版 u+w-e-f。

---

5. “权重是学出来的”与 u+w-e-f 为什么不矛盾?

在本项目里这是两层东西：

- **解析 baseline 权重（非学习）**：weights_dafner(img)
  - 由 u+w-e-f 从图像计算得出，用作 baseline / 参照 / residual 底座。
- **学习权重（Generator 输出）**：weights_gen = g(img)
  - 在 residual 模式下：
    - weights_gen = weights_dafner + Δ(img)
  - 即网络常学的是“在 baseline 上的修正”。

因此：

- u+w-e-f 给出一个强先验起点
- 神经网络学习如何在特定图像上偏离/修正它

---

6. 为什么需要 Evaluator E（代理）？

真实目标一般是：

`y_real = Metric(img, Decode(weights))`

但 Decode 包含 MST/离散 splice，整体对 weights 不可导。

Evaluator E 学的是一个可导近似：

`E(img, weights) ≈ Metric(img, Decode(weights))`

训练时：

- **先训练 E**：用程序算出来的真实标签监督（不是人工标注）
- **再训练 G**：通过 E 的梯度让 G 输出更优的 weights_gen

---

7. 两个核心训练风险点

7.1 E 的输入权重采样/覆盖（distribution matching）

如果 E 训练时见到的 weights 分布与推理/训练时 G 会产生的 weights_gen 分布不一致，E 就会在关键区域乱评分。

本项目使用的主要策略：

- 以 weights_dafner 为中心做扰动（翻符号、mix 噪声等）得到 weights_tgt
- 训练时将 weights_gen_detach 也喂给 E（在线分布对齐的一部分）

可进一步增强的方法（工程上常见）：

- replay buffer（缓存历史 weights_gen）
- curriculum（先弱扰动后强扰动）
- active sampling（刻意采样靠近 MST tie / 换边边界的区域）

7.2 代理偏差（surrogate gap）与“周期性真实校准”

风险：G 优化的是 E(img,weights)，但真实想要的是 Metric(img,Decode(weights))。

如果 E 有系统误差，G 会利用误差钻空子，出现：

- E 预测提升，但真实指标不提升甚至变差。

缓解方式（建议重点关注）：

- **周期性真实校准**：定期对当前 weights_gen 计算真实指标 y_real，回灌训练 E（DAGGER 风格数据聚合）。
- 交替/两时间尺度训练：让 E 更快跟上 G 的分布变化。
- 保守约束：限制 weights_gen 偏离 weights_dafner 的幅度（residual 本身就是一种保守）。

---

8. 2D 可复现脚本（工程闭环）

8.1 reproduce_final_2d.py

功能：

- 输入：随机图或 --img_path 指定图片（转灰度 resize 到 H×W）
- 解码：cover+MST+merge+cut
- 验证：
  - len(path)=H·W
  - len(set(path))=H·W
  - ∀t, ℓ1(path[t],path[t+1])=1
- 输出：
  - sfc.png（networkx 图结构）
  - ordered_curve.png（按访问顺序画折线，穿越 cell 中心，可叠加背景/网格/坐标）

8.2 generate_demo_images.py

生成可用于 --img_path 的 demo 图片到 img/，便于快速测试不同输入纹理下的曲线表现。

---

9. 3D 推广实验：2×2×2 cover + MST + matched-portal splice

为了“把 2D 思路直接放到 3D 看现象”，我们实现了一个最小 3D 原型：

- reproduce_final_3d.py

9.1 3D 合并卡壳现象（已复现）

对 D×H×W 体素网格（偶数维度），将其分块为 2×2×2 blocks：

- 每个 block 内先用 3-bit cyclic BRGC 初始化一个 8-cycle
- 在 block 对偶图上取 MST
- 沿 MST 尝试做 matched-portal splice

实验现象：在某些体数据/合并顺序下，可能在第一条树边就失败。

9.2 失败的本质原因

失败不是“没有 portal”，而是“portal 不匹配”：

- 两侧共享面各自确实存在 face 内边（portal edges）
- 但不存在 (u,v) 使得其映射 (φ(u),φ(v)) 恰好也是另一侧当前 cycle 的一条边

这就是 d≥3 时 face 之间耦合更强、portal 可用性动态耗尽/错配的具体表现。

9.3 诊断与可视化证据

reproduce_final_3d.py 支持：

- --debug_fail：打印 A 面 portal 及其映射到 B 后是否存在对应边
- 失败时生成：output/.../merge_fail.png
  - 3D 线框展示 A/B 两个立方体块、共享面顶点、两侧 portal edges、以及 A→B 映射后的未匹配虚线边

---

10. 结论与下一步（综合考虑）

10.1 2D：工程上闭环且可复现

2D 的 2×2 cover + MST + apply_mst 在本仓库是成熟、可复现、可验收（AC+覆盖）的。

10.2 3D：直接类比会遇到 matched-portal（需要额外自由度/策略）

将同样“block 内 cycle + 沿树 splice 合并”的思想推广到 3D，需要至少引入：

- block 内 cycle 的朝向/自同构自由度（Q3 的旋转/翻转）
- 或更复杂的 portal 选择策略/回溯

否则会出现“共享面有 portal 但不匹配”的合并失败。

10.3 训练侧的关键建议

在 E/G 框架下，除了模型结构本身，最影响可用性的往往是：

- E 的输入分布覆盖（围绕 baseline/围绕 G 的在线对齐）
- surrogate gap 的监测与周期性真实校准

这两点应当作为实现与论文论述中的重点风险控制点。